SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	S. LOHITHA	**Reg. No.:**	192524031
Date:	10/10/2026	**Duration:**	60 minutes

Code of Conduct

I S. Lohitha - 192524031 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Lohitha

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

1.1 Explain the problem in your own words.

The platform connects parking-space owners with drivers who need parking. Owners can list unused spaces, while drivers can search, reserve, and pay for suitable spaces online.

1.2 Identify the objectives of the proposed system.

- Register owners and drivers.
- List and search available parking spaces.
- Manage reservations and cancellations.
- Support digital payments and refunds.
- Provide ratings and transaction histories.

1.3 State the scope of the problem.

The system manages parking listings, availability, bookings, payments, cancellations, and reviews through a mobile or web application. Future versions may include AI recommendations and navigation.

2. Identify Key Stakeholders and Challenges

2.1 Identify all the stakeholders involved in the system.

The stakeholders are parking-space owners, drivers, system administrators, payment providers, property managers, security personnel, and the development team.

2.2 Explain their roles and responsibilities.

Owners list spaces and set prices. Drivers search and book spaces. Administrators manage users and disputes. Payment providers process transactions. Security personnel control access where required. Developers maintain the platform.

2.3 Discuss the major challenges faced by stakeholders.

Major challenges include finding available parking, incorrect listings, booking conflicts, payment failures, security concerns, privacy risks, and cancellation disputes.

3. Analyze the Current Approach

3.1 Describe the existing system or current process.

Drivers usually find parking by visiting locations, contacting owners, or using basic parking applications. Private parking spaces often remain unused because owners and drivers lack a common platform.

3.2 Identify its strengths and limitations.

Strengths: Simple arrangements, direct communication, and basic location information.

Limitations: Time-consuming searches, outdated availability, unclear prices, booking conflicts, and payment difficulties.

3.3 Analyze the problems or gaps that need to be addressed.

The existing approach lacks centralized booking, reliable availability information, transparent payments, and structured dispute management. The proposed platform addresses these problems through automated reservations and transaction records.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Design a software-based solution for the given problem.

Develop a mobile or web application where owners list spaces and drivers search, reserve, and pay for parking. The system manages availability, cancellations, notifications, and reviews.

4.2 Describe the major functional modules.

- User registration and login.
- Vehicle management.
- Parking-space listing.
- Search and filtering.
- Reservation management.
- Payment and refund processing.
- Ratings, reviews, and dispute management.
- Administration and transaction history.

4.3 Identify the functional and non-functional requirements.

Functional requirements: Registration, listing spaces, searching, booking, payments, cancellations, and reviews.

Non-functional requirements: Security, reliability, performance, usability, scalability, and maintainability.

4.4 Explain the application of software engineering principles.

- Modularity: Divides the system into independent modules.
- Scalability: Supports future growth.
- Maintainability: Simplifies updates and bug fixing.
- Reliability: Prevents duplicate bookings.
- Usability: Provides a simple interface.
- Security: Protects user information and transactions.

5. Justify the Chosen Methodology/Framework

5.1 Select an appropriate SDLC model or framework.

The selected methodology is Agile Scrum, which develops software through short, iterative cycles called sprints.

5.2 Justify why the methodology is suitable.

Agile Scrum supports incremental development, continuous user feedback, early testing, flexible requirements, and effective teamwork.

5.3 Explain how it supports successful development and maintenance.

The team gathers requirements, plans sprints, develops modules, tests features, and collects feedback. After deployment, regular updates and testing help maintain and improve the application.

6. Discuss Expected Outcomes and Limitations

6.1 Describe the expected benefits.

- Better utilization of unused parking spaces.
- Reduced parking search time.
- Additional income for owners.
- Convenient online reservations.
- Transparent payments and booking records.

6.2 Explain how the solution addresses the challenges.

Availability checks, secure payments, booking confirmations, clear cancellation policies, and dispute management reduce uncertainty and improve trust.

6.3 Discuss potential risks and limitations.

Incorrect listings, payment failures, unauthorized access, internet issues, low user adoption, and maintenance costs may affect the system. Verification, security measures, and regular testing can reduce these risks.

6.4 Suggest possible future enhancements.

AI-based parking recommendations.

Demand-based pricing.

Navigation integration.

Smart-gate access.

Tamil and other language support.

Parking-demand prediction using historical data.

Conclusion:

The Intelligent Parking Space Sharing Platform connects owners and drivers efficiently, reduces parking search time, and improves the use of existing parking spaces. Agile Scrum supports flexible development, continuous testing, and future improvements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability,	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.

	security, etc.).			
Methodology/ Framework Justification	Selects the most suitable SDLC/ model/fram	Suitable methodolo gy selected with	Methodolog y selected but justification is	Outcomes, limitations, or future enhancements
(10%)	ework and provides strong technical justification.	adequate justification.	weak.	are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25